

Brief Report: Shannon Entropy, Huffman Coding, and ZIP Files

This report explains Shannon entropy, Huffman coding, and how the provided encoder-decoder website relates these ideas to ZIP archives.

Shannon Entropy

Shannon entropy measures the average uncertainty or information in a data source. A source with many equally likely symbols has high entropy and is harder to compress. A source with repeated symbols has lower entropy and can usually be compressed more effectively.

$$H = - \sum p(x) \log_2 p(x)$$

In the website, entropy is calculated in the decoder by grouping the binary stream into byte-sized parts, counting how often each byte pattern appears, and applying this formula. This gives an entropy estimate for the bitstream being decoded.

Huffman Algorithm

Huffman coding is a lossless compression method. It counts symbol frequencies, builds a binary tree by repeatedly merging the two least frequent nodes, and assigns binary codes by following tree paths. Frequent symbols receive shorter codes, while rare symbols receive longer codes.

Your file implements this directly: it creates nodes with symbol, frequency, left child, and right child fields; sorts nodes by frequency; merges the two smallest nodes; then creates prefix codes using 0 for left paths and 1 for right paths.

Website File Analysis

The website supports text input and image input. For text, each character becomes a symbol. For image mode, the image is reduced to a small grayscale matrix and each grayscale value becomes a symbol such as Px_120. After frequencies are counted, the Huffman tree is drawn visually and the final compressed bitstream is displayed.

The decoder rebuilds the tree from the saved code dictionary, then reads the compressed bitstream bit by bit. Whenever traversal reaches a leaf node, the original symbol is restored. This correctly demonstrates the prefix-code property of Huffman coding, where no complete code is the beginning of another code.

Relation With ZIP Files

The website uses JSZip to export two files into an archive: bitstream.txt and schema.json. This means the ZIP file is used as a container for your custom Huffman-compressed data and its decoding dictionary. In this project, the Huffman encoding is performed by your JavaScript code before the data is placed inside the ZIP.

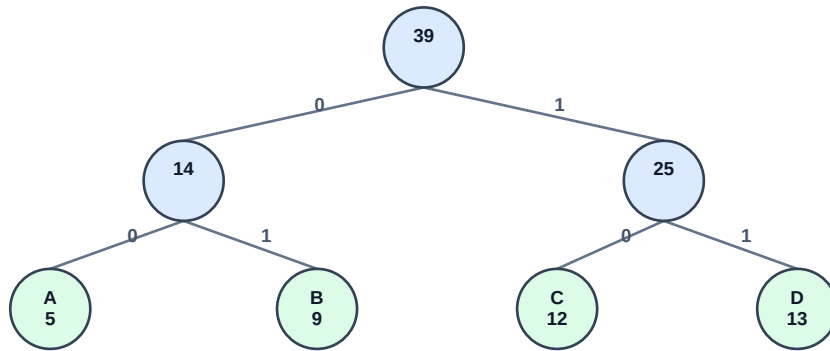
Standard ZIP compression is related but not exactly the same. Real ZIP files commonly use the DEFLATE method, which combines LZ77 pattern matching with Huffman coding. LZ77 replaces repeated strings with references, and Huffman coding then compresses the resulting symbols. Your project demonstrates the Huffman part clearly, while ZIP packaging is used to store and transfer the generated bitstream and schema.

Example Frequencies

Symbol	Frequency	Example Code
A	5	00
B	9	01
C	12	10
D	13	11

Example Huffman Tree

Frequencies: A=5, B=9, C=12, D=13. Smallest nodes are merged first.



Codes from this tree: A=00, B=01, C=10, D=11

Conclusion

The project demonstrates the core idea of lossless compression: repeated or frequent data can be represented with fewer bits. Shannon entropy explains the theoretical compression limit, Huffman coding builds practical prefix-free codes, and ZIP archives are connected because common ZIP compression methods also use Huffman coding internally.